ATHANASIOS GOURDOMICHALIS
Numerical Analysis Python Programs
Fixed-Point Iteration (General Iterative Method) 1.0

This document provides:

- **A short intro to Fixed-Point Iteration**
- **The program's problem statement (assignment prompt and requirements)**
- **The code description and implementation details.**

1. Introduction to Fixed-Point Iteration

- **Fixed-Point Iteration** (or General Iterative Method) is a **fundamental numerical technique** for finding **real roots** of a non-linear equation $\boldsymbol{f(x) = 0}$. By algebraically rearranging $f(x) = 0$ into the equivalent **fixed-point form**:

$$\boldsymbol{x = g(x)}$$

  we construct the **iterative sequence**:

$$\boldsymbol{x_{k+1} = g(x_k), \quad k = 0, 1, 2, \ldots}$$

- If $\boldsymbol{g}$ is **continuous** and the **generated sequence** converges, its limit $\boldsymbol{x^*}$ satisfies $\boldsymbol{x^* = g(x^*)}$, which is both **a fixed point of $g(x)$ and a root of $f(x)$**.
- **Convergence and Stability**:
  - **Contraction Mapping Condition**: By the Banach **Fixed-Point Theorem**, convergence is **guaranteed** in an **interval I** containing $\boldsymbol{x^*}$ if $\boldsymbol{g(x)}$ is a contraction, which holds if $\boldsymbol{|g'(x)| \leq \lambda < 1}$ for all $\boldsymbol{x \in I}$.
  - **Order of Convergence**: The asymptotic error satisfies $\boldsymbol{\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|} = |g'(x^*)|}$. When $\boldsymbol{0 < |g'(x^*)| < 1}$, the method exhibits **linear convergence** $(\boldsymbol{p = 1})$ with asymptotic error **constant** $\boldsymbol{C = |g'(x^*)|}$.

2. Problem Statement

- **Non-linear Equation:**

$$f(x) = x^3 + x - 1 = 0$$

- **Rearrangement Formulation:**

$$x = g(x), \quad \text{where } g(x) = \sqrt[3]{1 - x}$$

- **Requirements:**

1. **Fixed-Point Iteration:** We must implement $x_{k+1} = g(x_k)$ in **Python** starting from the **initial value** $x_0 = 0$, with stopping **tolerance** $|x_{k+1} - x_k| < 10^{-6}$ and a **maximum limit of 100 iterations**. We will store all **approximations** x_k into a list.

2. **Error Trajectory Plot:** We compute the **consecutive step errors** $e_k = |x_{k+1} - x_k|$ at **every iteration step** directly **inside the loop** and **produce a plot** with **matplotlib** showing **the error progression** across loops, **containing axis labels, a title, and a grid.**

3. **Rate & Constant Estimation**: Utilizing the formulas:

$$p_k = \frac{\ln(e_{k+1}/e_k)}{\ln(e_k/e_{k-1})} \quad \text{and} \quad C_k = \frac{e_{k+1}}{e_k^{p_k}}$$

we estimate the **numerical order** of **convergence** $\mathbf{p_k}$ and the **asymptotic error constant** $\mathbf{C_k}$.

3. Code Description & Implementation Details

- **Initial Setup & Helper Functions**:
  - We import **numpy** and **matplotlib.pyplot**, setting **`tol = 1e-6`** and **`max_iter = 100.`**
  - The mathematical rearrangement $g(x) = \sqrt[3]{1 - x}$ is **implemented using `np.cbrt(1.0 - x)`** to reliably **handle cube roots** when **evaluation hits zero or negative terms** without producing **floating-point domain errors**.

- **The Iteration Function (`FPI(x0)`):**
  - **Sequence Storage**: We store initial point x_0 and all **subsequent steps** inside **`x_k = [x0]`**.

- **Step Evolution:** At each iteration i, we compute $x_{next} = g(x_0)$ and **checks the stopping criterion** $|x_{next} - x_0| < tol$.
- **Error Visualization:**
  - The **consecutive step error** $e_k = |x_{k+1} - x_k|$ **is computed in each loop iteration** via `err.append(np.abs(x_next - x0))` and stored in the list `err`.
  - These **consecutive errors** are **then plotted** against the **iteration count** (`iterations = range(len(errors))`) to **visualize the convergence rate** of the **method**.

- **Asymptotic Calculation**:
  - The **last three non-zero errors** e_{k-1}, e_k, e_{k+1} are plugged into the **logarithmic formulas** (`np.log(…)`), yielding $p \approx 1$ and $C \approx 0.716$, matching theoretical $|g'(x^*)|$.